

9. Train a CNN Model for Handwritten Digit Recognition Using the MNIST Dataset

Train a CNN Model for Handwritten Digit Recognition Using the MNIST Dataset

Aim

To train and evaluate a Convolutional Neural Network (CNN) model for handwritten digit recognition using the MNIST dataset.

Introduction

Handwritten digit recognition is one of the most common applications of Deep Learning. It involves classifying handwritten digits (0–9) into their corresponding classes.

A Convolutional Neural Network (CNN) is well-suited for this task because it automatically extracts image features and learns patterns from training data. After training, the model can accurately recognize unseen handwritten digits.

Objective

In this practical, we will:

- Load the MNIST dataset.
- Preprocess the images.
- Train a CNN model.
- Evaluate the trained model.
- Predict handwritten digits.
- Visualize the prediction results.

Algorithm

1. Import the required libraries.
2. Load the MNIST dataset.
3. Normalize the pixel values.
4. Reshape the images.
5. Build the CNN model.
6. Compile the model.
7. Train the model.
8. Evaluate the trained model.
9. Predict handwritten digits.
10. Display prediction results.

```
In [1]: #Import Libraries
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras import datasets, layers, models
```

```
In [2]: # Load Dataset
```

```
(X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()

print("Training Images :", X_train.shape)
print("Testing Images :", X_test.shape)
```

Training Images : (60000, 28, 28)

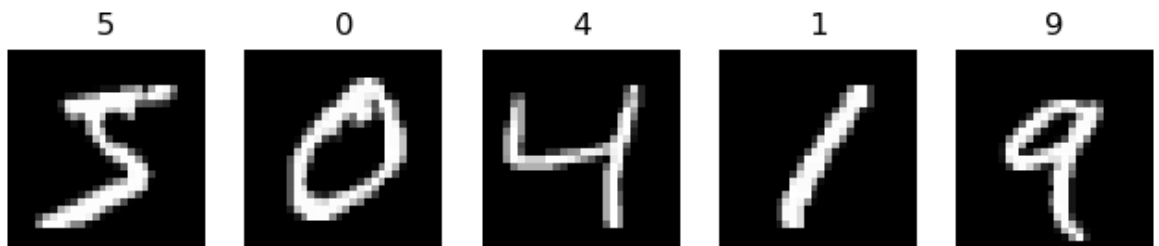
Testing Images : (10000, 28, 28)

```
In [3]: # Display Sample Images
```

```
plt.figure(figsize=(8,4))

for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(y_train[i])
    plt.axis('off')

plt.show()
```



```
In [4]: # Normalize Images
```

```
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape Images

X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)
```

In [5]: *# Build CNN Model*

```
model = models.Sequential([
    layers.Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(128,activation='relu'),
    layers.Dense(10,activation='softmax')
])
```

C:\Users\Smit\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [6]: `model.summary()`

Model: "sequential"

Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 26, 26, 32)	
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	
conv2d_1 (Conv2D)	(None, 11, 11, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	
flatten (Flatten)	(None, 1600)	
dense (Dense)	(None, 128)	
dense_1 (Dense)	(None, 10)	



Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

In [7]: *# Compile Model*

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

In [8]: *# Train Model*

```
history = model.fit(  
    X_train,  
    y_train,  
    epochs=5,  
    batch_size=64,  
    validation_split=0.2  
)
```

Epoch 1/5

750/750 ————— 22s 24ms/step - accuracy: 0.8770 - loss: 0.3992
- val_accuracy: 0.9797 - val_loss: 0.0649

Epoch 2/5

750/750 ————— 17s 22ms/step - accuracy: 0.9833 - loss: 0.0547
- val_accuracy: 0.9868 - val_loss: 0.0468

Epoch 3/5

750/750 ————— 17s 22ms/step - accuracy: 0.9871 - loss: 0.0394
- val_accuracy: 0.9877 - val_loss: 0.0431

Epoch 4/5

750/750 ————— 16s 21ms/step - accuracy: 0.9907 - loss: 0.0283
- val_accuracy: 0.9868 - val_loss: 0.0496

Epoch 5/5

750/750 ————— 17s 23ms/step - accuracy: 0.9937 - loss: 0.0198
- val_accuracy: 0.9895 - val_loss: 0.0375

In [9]: *# Evaluate Model*

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)  
  
print("Test Loss :", test_loss)  
print("Test Accuracy :", test_accuracy)
```

313/313 ————— 2s 7ms/step - accuracy: 0.9868 - loss: 0.0422
Test Loss : 0.030975809320807457
Test Accuracy : 0.9901000261306763

In [10]: *# Predict Test Images*

```
predictions = model.predict(X_test)  
  
predicted_labels = np.argmax(predictions, axis=1)
```

313/313 ————— 2s 7ms/step

```
In [11]: # Display First Five Predictions

plt.figure(figsize=(12,3))

for i in range(5):

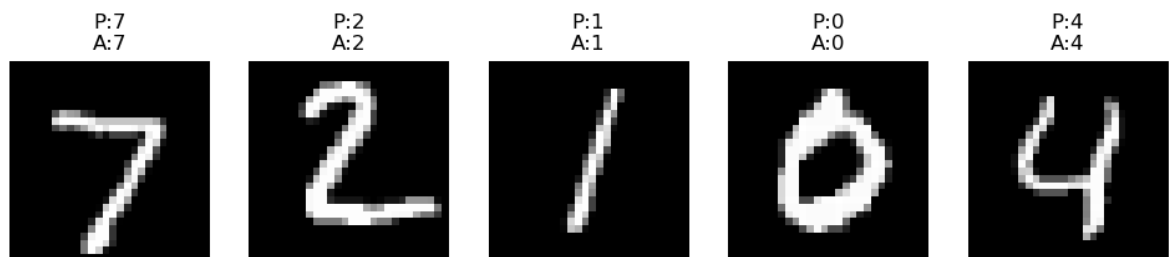
    plt.subplot(1,5,i+1)

    plt.imshow(X_test[i].reshape(28,28), cmap='gray')

    plt.title(f"P:{predicted_labels[i]}\nA:{y_test[i]}")

    plt.axis('off')

plt.show()
```



Expected Output

- The CNN model is successfully trained.
- The model achieves approximately **98–99%** accuracy on the test dataset.
- The first five test images are displayed with their predicted and actual labels.

Observation

- The CNN successfully learns handwritten digit patterns.
- The trained model correctly predicts most test images.
- CNN provides excellent accuracy for image classification tasks.

Result

The Convolutional Neural Network (CNN) model was successfully trained using the MNIST dataset. The trained model accurately recognized handwritten digits with a test accuracy of approximately **98–99%**.